



SENIOR .NET INTERVIEW SERIES

Async, Threading & Memory

async/await internals, synchronization, GC, and `Span<T>` — where senior candidates are separated from mid-level ones.

20 Real Interview Questions, Answered

Swipe through — save this for your next interview prep.

Q. What actually happens when you 'await' something?

A. The compiler rewrites the method into a state machine. At the await, if the task isn't complete, the method returns to its caller and registers a continuation; when the task finishes, the continuation resumes the method — on the captured context if one exists. No thread is blocked while waiting. That's the whole point: await frees the thread.

Q. async void vs async Task — why is async void dangerous?

A. Exceptions in an async Task method are captured in the task and observed on await. In async void there's no task — exceptions are rethrown on the SynchronizationContext and can crash the process. async void is uncatchable, untestable, and unawaitable. Reserve it strictly for event handlers, and wrap the body in try/catch.

Q. What causes an async deadlock, and how do you prevent it?

A. Blocking on async code — `.Result` or `.Wait()` — from a context that continuations need to resume on (UI thread, classic ASP.NET). The thread waits for the task; the task waits for the thread. Prevent it by going async all the way down. ASP.NET Core has no `SynchronizationContext`, so this specific deadlock disappears — but sync-over-async still starves the thread pool.

Q. When do you use `ConfigureAwait(false)`?

A. In library code, on every await: it skips capturing the context, avoiding deadlocks for consumers and reducing overhead. In ASP.NET Core application code it's a no-op (no context) but harmless. In UI code, don't use it when the next line touches the UI. It's about where the continuation runs — nothing else.

Q. lock vs SemaphoreSlim vs Interlocked — how do you choose?

A. Interlocked for single atomic operations on a field — cheapest. lock (Monitor) for short critical sections of synchronous code. SemaphoreSlim(1,1) when you must await inside the critical section — you can't await inside lock — or to throttle concurrency to N. Prefer Channel<T> or immutable data to eliminate the lock entirely.

Q. Explain the generational GC in .NET.

A. The heap is split into Gen0 (new objects), Gen1 (survivors), Gen2 (long-lived), plus LOH for objects over 85KB. Most objects die young, so Gen0 collections are frequent and cheap; Gen2 is rare and expensive. Senior insight: mid-life objects that survive to Gen2 then die are the worst pattern — caches with churn create exactly that.

Q. What is `Span<T>` and when does it matter?

A. `Span<T>` is a stack-only view over contiguous memory — array, `stackalloc`, or native — letting you slice and parse without copying or allocating. It matters on hot paths: parsing, formatting, buffers.

```
ReadOnlySpan<char> s = "2026-07-06".AsSpan();  
int year = int.Parse(s[..4]);    // zero allocations  
int month = int.Parse(s[5..7]);
```


Q. Task.Run — when is it right and when is it wrong in ASP.NET Core?

A. Wrong for wrapping I/O: it burns a thread pool thread to wait, doubling the cost — the request thread is already free thanks to `await`. Right for genuinely CPU-bound work you want off a latency-sensitive path, or for parallelism via `Parallel/PLINQ`. In web apps, heavy CPU work usually belongs in a background queue, not `Task.Run`.

Q. What are IHostedService and Channel<T> good for together?

A. A producer/consumer background pipeline: requests write work items to a bounded Channel and return immediately; a BackgroundService reads and processes them. Bounded capacity gives you backpressure instead of unbounded memory growth, and you get graceful shutdown via the stopping token — a pattern interviewers love to hear described end to end.

Q. How do you find and fix a memory leak in a .NET service?

A. Capture dumps (dotnet-gcdump / dotnet-dump) at intervals, diff object counts, and walk GC roots to find who's holding them. Usual suspects: static event handlers never unsubscribed, caches without eviction, captured closures in long-lived delegates, and singletons resolving scoped services. Fix the root, prove it with a follow-up dump under load.

Q. Requests hang for 30s under load, CPU is at 5%, and thread count is slowly climbing. Diagnose.

A. Textbook thread pool starvation: blocked threads (sync-over-async, blocking I/O, or lock convoys) while the pool injects roughly one thread per second — hence low CPU, slow climb, queued work stalling. Confirm with dotnet-counters (queue length) and a dump showing threads parked in Wait. Fix the blocking calls; scaling out just delays the collapse.

Q. Parallel.ForEach vs Task.WhenAll vs Parallel.ForEachAsync — pick for calling 500 HTTP endpoints.

A. Parallel.ForEach is wrong: it's for CPU-bound work and would block threads on I/O. Naive Task.WhenAll over 500 tasks fires everything at once — connection exhaustion and rate limits. Parallel.ForEachAsync with MaxDegreeOfParallelism gives you async I/O with bounded concurrency — the right default. Same effect achievable with a SemaphoreSlim gate around WhenAll.

Q. Your cache uses Dictionary with a lock. Reviewer suggests ConcurrentDictionary. What's your analysis?

A. ConcurrentDictionary removes the global lock, but GetOrAdd's factory can run multiple times under race — expensive loaders need Lazy<T> wrapping. If it's really a cache, both are wrong: no eviction means a memory leak — use IMemoryCache with size limits. The senior move is questioning the requirement, not just swapping the collection.

Q. A BackgroundService stopped processing three days ago and nobody noticed. What do you change?

A. Immediate: wrap the loop so one poison item can't kill it, log fatally on exit. Structural: heartbeat metric per loop iteration with an alert on absence — silence must page someone; queue lag alerts as a second net. And ask why the host didn't die: in .NET 8+ an unhandled BackgroundService exception stops the host by default — someone probably suppressed it.

Q. When would you set gcServer=true, and when has it backfired?

A. Server GC: per-core heaps and parallel collection — right for throughput-oriented services on dedicated cores, and default for ASP.NET Core. Backfires in containers with small CPU/memory limits: per-core heaps balloon RSS and you get OOMKilled, or many services co-located each claim per-core heaps. There, Workstation GC or DATAS (adaptive, .NET 8+) is the better fit.

Q. Explain a real use you've had for CancellationToken beyond 'pass it along'.

A. Linked token sources: combining the request's token with a per-operation timeout via `CreateLinkedTokenSource`, so cancellation fires on whichever comes first. Also: cooperative cancellation checkpoints in a long CPU loop (`ThrowIfCancellationRequested` per batch), and honoring token in cleanup to distinguish 'user left' from 'we timed out' for correct logging. Propagation alone is table stakes.

Q. You replaced string concatenation with StringBuilder and gained nothing. Why?

A. Because allocation wasn't the bottleneck — or the compiler had already optimized it: constant concatenation folds at compile time, and interpolated strings now use low-allocation handlers. StringBuilder pays off in loops building large strings, not in three-part concatenations. The lesson interviewers want: profile before optimizing; dogma about 'strings are slow' isn't measurement.

Q. What's the difference between `IAsyncEnumerable<T>` and returning `Task<List<T>>`, practically?

A. `IAsyncEnumerable` streams items as they're produced — first result arrives immediately, constant memory, cancellation between items, and ASP.NET Core can stream it as NDJSON/SSE. `Task<List<T>>` materializes everything first: simpler, but latency equals the slowest item and memory equals the full set. Choose streaming for large or slow sequences, lists for small bounded results.

```
await foreach (var row in db.ReadRowsAsync(ct))
{
    yield return Transform(row); // O(1) memory
}
```

Q. A dump shows 2GB of strings held by a static ConcurrentDictionary used for 'caching'. Walk through the fix.

A. Root cause: unbounded cache keyed by something high-cardinality (URLs, user IDs) — every entry is GC-rooted forever. Fix: IMemoryCache with SizeLimit and per-entry size, TTL matched to data volatility, and metrics on hit rate and entry count. Then audit for the same pattern elsewhere — static collections that only grow are the most common leak in .NET services.

Q. How do you make a hot serialization path allocation-free, concretely?

A. System.Text.Json with source generation removes reflection; write into pooled buffers — `ArrayPool<byte>` or `IBufferWriter` via `Utf8JsonWriter` — instead of intermediate strings; parse with `Utf8JsonReader` over `ReadOnlySpan<byte>`. Measure with `[MemoryDiagnoser]` before and after. And validate it matters: shaving allocations on a path called once a second is engineering theater.

That's a wrap.

20 Async, Threading & Memory questions that actually come up in senior .NET interviews — with the answers experienced engineers give.

NEXT IN THE SERIES

Runtime & Performance

Found this useful?

Follow for the rest of the series,
and save this post for your interview prep.